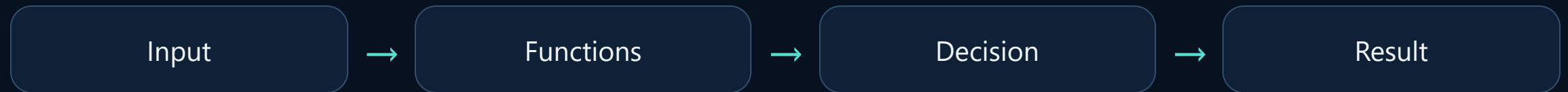


Just-enough Python.

Read the workflow. Test the decisions. Change one rule safely.



**TODAY'S
OUTCOME**

Understand the moving parts.

Read

Variables, strings, lists,
dictionaries.

Organize

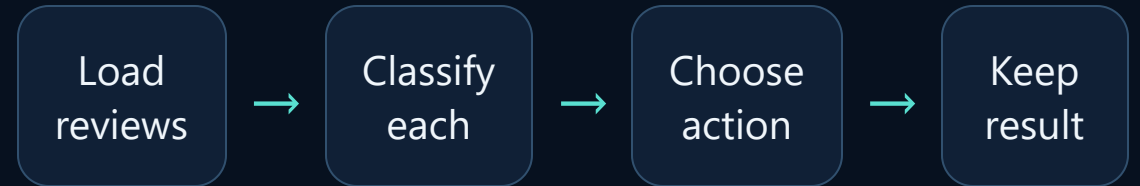
Functions, parameters,
return values, type hints.

Control

Loops, conditions, errors,
tests.

READABLE
WORKFLOW

Read code as a story.



**VARIABLES
AND STRINGS**

Give values useful names.

```
review = "Stopped  
charging after two days"  
sentiment = "negative"  
answered = 0
```

= assigns

Put this value
under this name.

**Quotes
mean text**

A Python string
holds characters.

F-
STRINGS

Build text from values.

```
prompt = f"Classify this  
review: {review}"  
print(f"Answered {answered} of  
{len(questions)}")
```

Insert data into text. Do not execute
untrusted text as code.

FUNCTIONS

A function is a named recipe.

```
def classify(review):  
    answer = ask(...)  
    return  
extract_json(answer)  
  
result = classify(review)
```

TYPE
HINTS

Make the contract visible.

```
def classify(review: str)
  -> dict:
    ...

def
flag_if_complaint(result:
dict) -> str:
    ...
```

Hints explain intent. Important runtime boundaries still need validation.

DICTIONARIES

Use labels to reach values.

```
result = {  
    "sentiment": "negative",  
    "topic": "charging",  
    "summary": "Stopped charging  
quickly"  
}  
  
print(result["sentiment"])
```

LISTS

Keep many items in order.

```
reviews = [  
    "Arrived early and works.",  
    "Stopped charging.",  
    "Setup is confusing."  
]
```

```
results = []  
results.append(result)
```

LOOPS

Write the process once.

```
for review in reviews:  
    result =  
    classify(review)  
  
results.append(result)
```

Three items or three hundred: the loop shape stays the same. Outside calls still need limits.

CONDITIONS

Choose exactly one path.

```
if sentiment ==  
"negative":  
    return "send to  
support"  
elif sentiment ==  
"unclear":  
    return "human  
check"  
else:  
    return "ok"
```

= assigns. ==
compares.

PUT IT
TOGETHER

One clear step per line.

```
for review in reviews:  
    result = classify(review)  
    action =  
flag_if_complaint(result)  
results.append(result)  
print(result, action)
```

Classification plus action.

positive | delivery | Arrived
early

action: ok

negative | charging | Stopped
charging

action: COMPLAINT – send to
support

unclear | product | No clear
opinion

action: human should check

Trace main into two functions.

classify

Outside model call and parsing.

flag

Local routing rule.

main

List, loop, collect, print.

LOCAL UNIT
TEST

Test decisions without a model.

positive

→ ok

negative

→ support

unclear

→ human
check

PROVIDER
PATH

Run live only when access works.

```
python phase-1-foundations/day-  
03/labs/  
lab-3.A-review-sorter-with-  
functions/solution/sorter_functions.py
```

Three reviews mean three
provider calls in this
implementation.

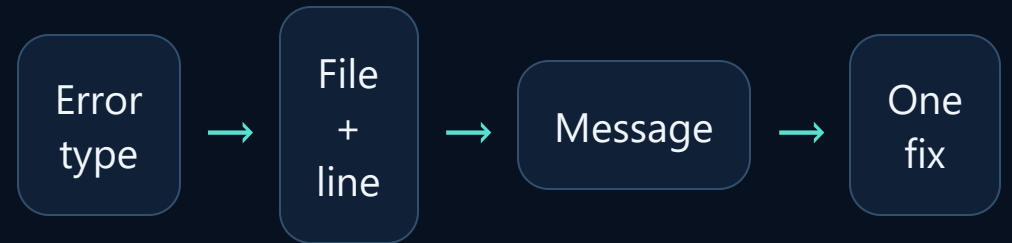
LAB
3.B

Skip bad input before a call.

```
for number, question in enumerate(questions,  
start=1):  
    if not question.strip():  
        print("skipped")  
        continue  
    answer = ask(question)  
    answered += 1
```

DEBUGGING

Read the first useful error.



**PRODUCTION
BOUNDARY**

A loop can multiply outside risk.

Calls

Count them.

Cost

Set a
budget.

Latency

Set
timeouts.

Failure

Handle each
item.

COMPLETION
GATE

**Day 3 is complete
when you can change
the workflow safely.**

- ✓ Explain every function input and output.
- ✓ Local decision tests pass.
- ✓ The batch skips blank input.
- ✓ One error is read, fixed, and rerun.
- ✓ One routing rule changes in one function.

Next: APIs,
secrets, and
failures.